SIPaaS
Complete Test Guide

API-Based -- No Softphone Required

From laptop restart to live call -- using only curl + Docker

Platform: SIPaaS	Carrier: Vocallabs (160.30.71.89:3300)	Prefix: 45454

What This Guide Covers

This guide walks you through testing the SIPaaS platform entirely using REST API calls and Docker CLI commands. No SIP softphone is needed. You can trigger real outbound PSTN calls directly via the API.

```
>> API BASE URL : http://localhost:3000
ARI URL : http://localhost:8088
PORTAL URL : http://localhost:8080
```

STEP 0

Start Docker Desktop

Must be running before any containers can start

1. Open Docker Desktop from the Start menu or taskbar icon.
2. Wait until the whale icon in the system tray turns solid (not animated).
3. The Docker Desktop window should show: "Docker Desktop is running"

If Docker Desktop is slow to start, give it 60-90 seconds.

WSL2 backend must be enabled: Settings > General > Use WSL2 based engine.

STEP 1

Open PowerShell and Navigate

All commands run from the sipaas project directory

Open Windows PowerShell (or Windows Terminal) and run:

COMMAND

```
cd C:\Users\Lenovo\sipaas
```

EXPECTED OUTPUT

```
PS C:\Users\Lenovo\sipaas>
```

STEP 2

Start All Containers

Brings up the full SIPaaS stack

COMMAND

```
docker compose --profile sip up -d
```

Wait about 30 seconds after this command. MySQL needs time to initialize.

First run may take longer as Docker pulls base images.

EXPECTED OUTPUT

```
Container sipaas-db Started

Container sipaas-redis Started
```

```
Container asterisk Started

Container kamailio Started

Container rtpengine Started

Container sipaas-api Started

Container sipaas-portal Started

Container sipaas-billing Started
```

STEP 3

Verify All Containers Are Running

Check health status of every container

COMMAND

```
docker compose ps
```

All containers should show "Up" or "healthy". If any show "Exit" or "Restarting", check their logs:

DEBUG COMMANDS

```
docker logs --tail=30

# Examples:

docker logs sipaas-api --tail=30

docker logs asterisk --tail=30

docker logs kamailio --tail=30
```

STEP 4

Check Vocallabs SIP Registration

Verify upstream carrier is connected

COMMAND

```
docker exec asterisk asterisk -rx "pjsip show registrations"
```

EXPECTED OUTPUT

```
==========================================================================
```

```
vocallabs-registration/sip:160.30.71.89:3300 vocallabs-auth Registered
```

If status shows "Rejected" or "Failed":

- Check your internet connection
- Verify VOCALLABS_PASS in .env file
- Try: docker compose restart asterisk then wait 10 seconds

STEP 5

Register a Test Account via API

Create your customer account -- no UI needed

The REST API is running at http://localhost:3000

COMMAND (use ^ for line continuation in Windows PowerShell)

```
curl -s -X POST http://localhost:3000/v1/auth/register ^

-H "Content-Type: application/json" ^

-d "{\"email\":\"test@sipaas.local\",\"password\":\"Test@1234\",\"name\":\"Test User\"}" ^

| python -m json.tool
```

EXPECTED OUTPUT

```
{

"id": 1,

"email": "test@sipaas.local",

"name": "Test User",

"api_key": "abc123def456...",

"api_secret": "xyz789..."

}
```

If you get 409 Conflict: email already registered. Skip to Step 6 and just login.

The api_key and api_secret can be used for programmatic API access.

STEP 6

Login and Get JWT Token

Required for all subsequent API calls

COMMAND

```
curl -s -X POST http://localhost:3000/v1/auth/login ^

-H "Content-Type: application/json" ^

-d "{\"email\":\"test@sipaas.local\",\"password\":\"Test@1234\"}" ^
```

```
| python -m json.tool
```

EXPECTED OUTPUT

```
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsImlhdCI6...",
"customer": {
"id": 1,
"email": "test@sipaas.local",
"balance": "0.0000"
}
}
```

IMPORTANT: Copy the token value. Save it as a PowerShell variable:

$TOKEN = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

Every API call from here uses: -H "Authorization: Bearer $TOKEN"

Token expires in 24 hours -- re-login if you get 401 errors.

STEP 7

Top Up Account Balance

Kamailio rejects calls with 402 if balance is zero

COMMAND

```
curl -s -X POST http://localhost:3000/v1/billing/topup ^
-H "Content-Type: application/json" ^
-H "Authorization: Bearer $TOKEN" ^
-d "{\"amount\":100,\"payment_ref\":\"test-topup-001\"}" ^
| python -m json.tool
```

EXPECTED OUTPUT

```
{
"balance": 100,
"currency": "INR"
```

```
}
```

Balance is stored in MySQL and synced to Redis. Kamailio reads from Redis key: sipaas:balance:{customer_id}

STEP 8

Create a SIP Trunk

A trunk = a SIP identity for making calls

COMMAND

```
curl -s -X POST http://localhost:3000/v1/trunks ^

-H "Content-Type: application/json" ^

-H "Authorization: Bearer $TOKEN" ^

-d "{\"name\":\"My Test Trunk\",\"max_channels\":2}" ^

| python -m json.tool
```

EXPECTED OUTPUT

```
{

"id": 1,

"sip_username": "c1_1748534921543",

"sip_password": "a1b2c3d4e5f67890abcdef...",

"sip_domain": "49.39.175.29",

"sip_server": "49.39.175.29",

"sip_port": 5060,

"auth_type": "digest",

"max_channels": 2

}
```

SAVE the "id" field -- this is your trunk_id for Step 9.

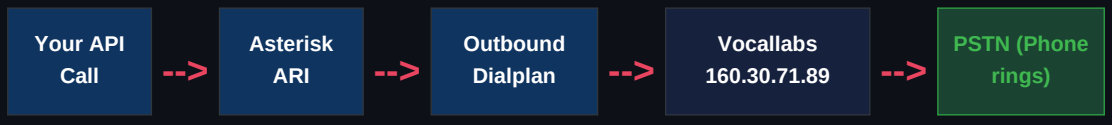
The trunk is now provisioned in Kamailio subscriber table.

max_channels: 2 means this trunk allows 2 simultaneous calls.

STEP 9

Make a Test Call via API

THE MAIN TEST -- triggers real outbound PSTN call



COMMAND

```
curl -s -X POST http://localhost:3000/v1/calls/originate ^

-H "Content-Type: application/json" ^

-H "Authorization: Bearer $TOKEN" ^

-d "{\"from\":\"919228130351\",\"to\":\"+919142436879\",\"trunk_id\":1}" ^

| python -m json.tool


# Replace +919142436879 with the actual destination number

# Replace trunk_id:1 with the id from Step 8
```

EXPECTED OUTPUT


```
{

"call_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",

"status": "originating",

"from": "sip:919228130351@49.39.175.29",

"to": "sip:+919142436879@49.39.175.29"

}
```


What happens internally when you hit this API:

1. API validates token, checks balance > 0, verifies trunk ownership
2. API calls Asterisk ARI: POST http://localhost:8088/ari/channels
3. Asterisk dialplan strips + from +919142436879 -> 919142436879
4. Dialplan prepends prefix 45454 -> sends 454549191424368 79 to Vocallabs
5. Vocallabs receives INVITE at 160.30.71.89:3300 and routes to PSTN
6. If Vocallabs PSTN routing is active, the destination phone rings

STEP 10

Check Active Channels

Verify the call is live and in progress

Check via the API:

API COMMAND

```
curl -s http://localhost:3000/v1/calls/active ^

-H "Authorization: Bearer $TOKEN" ^

| python -m json.tool
```

EXPECTED OUTPUT

```
{ "active_channels": 1 }
```

Or check Asterisk directly:

ASTERISK CLI

```
docker exec asterisk asterisk -rx "core show channels"
```

EXPECTED OUTPUT

```
Channel Location State Application(Data)

PJSIP/vocallabs-trunk-00000 +919142436879@... Up Dial()

1 active channel

1 active call
```

STEP 11

Watch Live SIP Trace

See the full SIP signaling in real time

Open a SECOND PowerShell window and run:

IN A SECOND POWERSHELL WINDOW

```
# Enable SIP packet logging

docker exec asterisk asterisk -rx "pjsip set logger on"


# Stream live Asterisk logs
```

```
docker logs asterisk -f
```

EXPECTED OUTPUT

```
-- Called +919142436879@outbound

-- Executing [+919142436879@outbound:3] Set DNIS=45454919142436879

-- Sending DNIS: 45454919142436879 via PJSIP/vocallabs-trunk

-- Called PJSIP/45454919142436879@vocallabs-trunk

-- PJSIP/vocallabs-trunk-00000000 answered

-- Channel PJSIP/vocallabs-trunk joined simple_bridge
```

Number format breakdown:

Input to API : +919142436879

After strip + : 919142436879

After prefix : 45454919142436879 <-- this is what Vocallabs receives

Caller ID shown: 919228130352 (Vocallabs CLI shown to destination)

STEP 12

Check CDR -- Call Detail Records

Verify billing and call history

COMMAND

```
curl -s "http://localhost:3000/v1/calls/cdr?limit=5" ^

-H "Authorization: Bearer $TOKEN" ^

| python -m json.tool
```

EXPECTED OUTPUT

```
{

"cdrs": [

{

"id": 1,

"src": "919228130351",

"dst": "919142436879",
```

```
"start_time": "2026-05-12T10:00:00.000Z",

"disposition": "ANSWERED",

"duration": 30,

"billsec": 28,

"cost": "0.2100",

"rate": "0.45"

}

]

}
```


*cost = billsec/60 * rate_per_min = 28/60 * 0.45 = 0.21 INR*

Alternative Methods

ALT A -- Direct Asterisk CLI Call (bypasses API + Kamailio)

Quick testing without auth, balance check, or CDR writing:

```
ASTERISK CLI

docker exec asterisk asterisk -rx ^

"channel originate Local/+919142436879@outbound application Wait 30"


# Then watch the result:

docker logs asterisk --tail=20
```

ALT B -- Direct ARI Call (bypasses API layer only)

Calls Asterisk ARI directly -- skips the Node.js API but goes through dialplan and Vocallabs:

```
DIRECT ARI COMMAND

curl -s -X POST "http://localhost:8088/ari/channels" ^

-u "ariuser:AriPassword!" ^

-H "Content-Type: application/json" ^

-d "{\"endpoint\":\"PJSIP/45454919142436879@vocallabs-trunk\",\"callerId\":\"91922
8130352\",\"timeout\":30,\"context\":\"from-ari\",\"extension\":\"+919142436879\",
\"priority\":1}" ^

| python -m json.tool
```

ARI credentials: ARI_USER=ariuser / ARI_PASS=AriPassword! (set in .env)

Note: the 45454 prefix is already included in the endpoint string above.

Troubleshooting

PROBLEM	CAUSE	FIX
API returns 401 Unauthorized	Wrong or expired JWT token	Re-run Step 6 login, copy new token into $TOKEN
API returns 402 Payment Required	Account balance is zero	Run Step 7 topup: POST /v1/billing/topup
API returns 403 Forbidden	trunk_id does not belong to your account	Run GET /v1/trunks to list your trunks, use correct id
API returns 503 Service Unavailable	Max concurrent channels reached for trunk	Increase via: PATCH /v1/trunks/:id with max_channels
Call drops in under 1 second	Vocallabs 200 OK + immediate BYE	Outbound PSTN not activated on Vocallabs account -- fix server side
sipaas-api container exits	DB or Redis not ready yet	Wait 30s, run: docker compose up -d again
pjsip show: Registration Failed	Wrong Vocallabs credentials or account suspended	Check VOCALLABS_PASS in .env, verify on Vocallabs portal
200 OK then immediate BYE	Vocallabs cannot reach your RTP/media IP	Open ports 10000-20000 UDP on firewall/router
API cannot reach Asterisk ARI	ASTERISK_ARI_URL misconfigured	Check docker-compose.yml ASTERISK_ARI_URL env var
Kamailio exits with uacreg error	DB schema version mismatch (expected v5, got v2)	Run the ALTER TABLE migration for uacreg table

Handy Commands Cheat Sheet

Container Management

```
docker compose ps # status of all containers

docker compose --profile sip up -d # start all SIP containers

docker compose restart asterisk # restart single container

docker compose logs -f --tail=20 # tail all logs together

docker logs asterisk --tail=50 # asterisk logs only
```

Asterisk CLI

```
docker exec -it asterisk asterisk -rvvv

docker exec asterisk asterisk -rx "pjsip show registrations"

docker exec asterisk asterisk -rx "core show channels"

docker exec asterisk asterisk -rx "pjsip set logger on"

docker exec asterisk asterisk -rx "dialplan reload"

docker exec asterisk asterisk -rx "module reload res_pjsip.so"
```

API Quick Tests

```
# Check balance

curl -H "Authorization: Bearer $TOKEN" http://localhost:3000/v1/billing/balance

# List trunks

curl -H "Authorization: Bearer $TOKEN" http://localhost:3000/v1/trunks

# Call history

curl -H "Authorization: Bearer $TOKEN"
"http://localhost:3000/v1/calls/cdr?limit=10"

# Active channels

curl -H "Authorization: Bearer $TOKEN" http://localhost:3000/v1/calls/active
```

Redis and MySQL

```
# Check customer balance in Redis (replace 1 with customer id)

docker exec sipaas-redis redis-cli -a RedisPass789! get sipaas:balance:1
```

```
# Check active channel count

docker exec sipaas-redis redis-cli -a RedisPass789! get sipaas:channels:1

# MySQL quick query

docker exec sipaas-db mysql -u sipaas -pStrongDBPassword123! kamailio ^

-e "SELECT id,name,sip_username,status FROM trunks LIMIT 5;"
```

Kamailio

```
docker exec kamailio kamctl dispatcher show # show Asterisk targets

docker exec kamailio kamctl ul show # show registered SIP users

docker logs kamailio --tail=30 # Kamailio logs
```

Current System Status

Component	Status	Notes
Stack / Containers	WORKING	All containers start and remain stable
Kamailio	WORKING	uacreg schema fixed, dispatcher active
Asterisk	WORKING	PJSIP registered, ARI on 8088, AMI on 5038
Vocallabs Registration	REGISTERED	160.30.71.89:3300 -- account 1769066634611
SDP / RTP IP	CORRECT	Advertises 49.39.175.29 (laptop public IP)
Outbound Prefix	CORRECT	45454 prepended -- sends 45454919XXXXXXXXXX
PSTN Routing	PENDING	Vocallabs sends 200 OK + immediate BYE
Fix Required	Vocallabs side	Activate outbound PSTN on account 1769066634611